Devoid Viewer

Codebase & Installer Documentation

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase, and documents the WiX installer. It is written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	5
4	loader.rs — image decoding	7
5	main.rs — application core	9
6	Threading & asynchronous design	18
7	Key design decisions & trade-offs	20
8	Caveats & known limitations	22
9	Installer — main.wxs	23
10	Glossary of Rust concepts used	26

1. Project Structure Overview

The project follows the standard Cargo layout:

File	Responsibility
`Cargo.toml`	Project name, version, and all external dependencies.
`build.rs`	Build script — embeds a Windows icon and metadata via winres.
`src/main.rs`	Event loop, app state, input handling, pixel renderer, window management.
`src/loader.rs`	Image decoding for all supported formats. Owns Frame and DecodedImage.
`src/nav.rs`	Directory scanning, human-sorted sibling list, next/prev navigation.
`wix/main.wxs`	WiX installer definition — features, file associations, shortcuts.
`devoid.ico`	Application icon used by both the executable and the installer.

2. Cargo.toml — Dependencies Explained

Each crate was chosen deliberately for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB, compared to 200+ crates and ~9 MB with a full GUI toolkit.

Windowing: winit

winit is the de-facto standard Rust crate for OS windows and input events. It handles the event loop, keyboard, mouse, window resize, drag-and-drop, and monitor queries. It has no rendering opinion. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires.

Pixel display: softbuffer

softbuffer provides a Vec<u32> mapping directly to the window's pixels. You write colours and call present(). No GPU pipeline, no shader compilation. On Windows it uses a GDI DIB section, compositor-accelerated without requiring a GPU context.

> ■ *Known limitation: the DWM compositor fills every new window with white before the application paints. This is a fundamental Win32 property. Mitigated by painting the softbuffer gray synchronously before the event loop starts, and hiding the window until the first image is ready.*

Native dialogs: windows-sys

windows-sys is a zero-overhead Windows API binding. Used for MessageBoxW (delete/error dialogs) and SystemParametersInfoW(SPI_GETWORKAREA) (exact taskbar exclusion). Target-specific — does not affect non-Windows builds.

Image decoding: image

The **image** crate supports BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. Features enabled selectively. GIF animation uses image::codecs::gif::GifDecoder and IntoFrames.

JPEG XL: jxl-oxide

JPEG XL is not yet in the image crate. **jxl-oxide** is a pure-Rust JXL decoder using the low-level streaming API.

SVG: resvg + usvg + tiny-skia

- **usvg** parses and normalises the SVG scene tree.
- **resvg** renders into a tiny-skia Pixmap.
- **tiny-skia** rasterises to premultiplied RGBA, which we unpremultiply.

Channel communication: crossbeam-channel

Faster and more ergonomic than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds devoid.ico and version metadata into the .exe. Zero runtime cost.

3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

struct DirNavigator

- **files: Vec<PathBuf>** — sorted sibling list.
- **index: usize** — position of the current file.

Derives **Default** — auto-generated empty initialiser.

fn set_current(path)

Called on every image open. Rebuilds the sibling list only when the directory changes. Falls back to a single-entry list if the path is not found in the scan.

fn rebuild(dir)

Reads the directory, filters for supported files, sorts using **human_sort** so img2 < img10 rather than img10 < img2.

fn next() / fn prev()

Return None if fewer than 2 images exist — navigation on a single-image directory is a no-op, preventing a redundant reload. Both wrap around and use checked_sub(1) for safe unsigned arithmetic.

fn peek_next() / fn peek_prev()

Read-only lookahead used by preload_adjacent(). Also returns None for single-image directories.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds.

fn human_sort / fn consume_number

Digit runs compared numerically: consume_number() reads the full run as u64, ensuring img2 < img10.

4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns **DecodedImage** directly — no conversion needed in main.rs.

struct Frame

pixels: Vec<u8> — straight RGBA8, row-major. **delay_ms: u32** — frame duration (0 for non-animated formats).

struct DecodedImage

One or more frames sharing the same dimensions. **is_animated()** — true when frames.len() > 1. **frame(idx)** — reference to a specific frame. **single()** — private convenience constructor.

fn load_image(path) → LoadResult

Dispatches by lowercased extension: svg → load_svg, jxl → load_jxl, gif → load_gif, everything else → load_standard.

fn load_standard — BMP, PNG, JPEG, WebP, TIFF, ICO

image::open() decodes and normalises to RGBA8 via into_rgba8().

fn load_gif — Animated GIF

Uses GifDecoder with AnimationDecoder. Dimensions read before consuming into frames — single file open. Per-frame delay from frame.delay().numer_denom_ms() with 20ms minimum and 100ms fallback.

fn load_jxl — JPEG XL

- BufReader for efficient I/O.
- request_color_encoding(srgb(...)) forces sRGB output.
- render_frame(0) decodes frame 0.
- Stream yields interleaved f32. Channels 1/2/3/4 expanded to RGBA u8 via f32_to_u8 (clamps then scales).

fn load_svg — SVG

- usvg parses and normalises the scene tree.
- resvg renders to a tiny-skia Pixmap (premultiplied RGBA).
- Unpremultiply: divide each channel by alpha. Zero-alpha pixels zeroed.

5. main.rs — Application Core

Contains the winit event loop, all application state, and the pixel renderer. No widget toolkit, no font renderer, no GPU pipeline.

5.1 const BG: u8 = 42

Background grey for the blank state and alpha compositing. Used in render(), composite(), and the pre-loop synchronous gray paint.

5.2 enum Msg

Four variants sent from background threads to the UI thread via the crossbeam channel:

- **Loaded** — primary decode complete: path, image, resize flag.
- **Preloaded** — speculative decode for the cache.
- **Error** — decode failed: path and error string.
- **Deleted** — async deletion result: Ok(()) or Err(String).

All contained types are Send automatically.

5.3 struct WakeUp / type Proxy

WakeUp is a zero-size marker injected by background threads via proxy.send_event(WakeUp) to wake the sleeping event loop. **Proxy** is a type alias for the verbose EventLoopProxy generic.

5.4 struct View<'a>

A snapshot of what the renderer needs: image reference, current frame index, and pan offset. App::view() produces one each frame. The 'a lifetime ensures View cannot outlive App. Passing View to render() keeps the signature to four arguments and prevents the renderer from mutating app state. Note: zoom was removed entirely — the viewer always uses fit-to-window scale.

5.5 enum OpenMode

FirstImage — resize the window to the image. **Navigate** — keep the existing window size.

5.6 struct App — fields

- **current_image** — not cleared when navigation starts. The previous image stays visible until the new one is decoded.
- **frame_index / next_frame_at** — GIF animation state.
- **pan / drag_start / pan_origin** — pan state. Reset on each new image. No zoom field.
- **loading_path** — path being decoded. Used to discard stale channel messages if the user navigated away.
- **preload_cache** — decoded neighbours in CPU memory.
- **deleting** — prevents double-deletion in flight.

- **pending_center** — triggers centering on the event after a resize has been applied (two-event sequence).

5.7 fn open_image()

Cache hit → apply_image directly. Otherwise sets loading_path, updates the navigator, spawns a background thread that calls load_image(), posts a Msg, and fires WakeUp.

5.8 fn apply_image() → Option<(u32, u32)>

Stores the image, resets pan, resets animation, updates the navigator, starts preloading. Returns Some((w,h)) when the window should resize — a return value, not a side-effect field.

5.9 fn clear_image()

Resets current_image, current_path, then delegates to reset_pan() and reset_animation() — one place for each reset's logic.

5.10 fn reset_animation() / fn tick_animation()

reset_animation() sets frame_index=0 and computes next_frame_at. Called from both apply_image() and clear_image().

tick_animation() checks the deadline, advances frame_index, schedules the next deadline. Uses if-let — no unwrap().

5.11 fn reset_pan() / fn start_drag() / fn update_drag() / fn end_drag()

reset_pan() zeroes pan and clears drag_start. Called on every new image and on Backspace/Home/middle-click.

update_drag() computes new pan from cursor delta. Returns true if dragging, so the caller can request a redraw without a separate is_some() check.

5.12 fn navigate(forward: bool)

Calls navigator.next() or prev() — both return None when fewer than 2 images exist, making navigation on a single-image directory a no-op.

5.13 fn request_delete() / fn on_deleted()

request_delete() sets deleting=true and spawns a thread for remove_file(). The UI never blocks on deletion I/O.

on_deleted() clears the flag, removes from navigator, clears the preload cache, navigates to the next sibling or clears the view.

5.14 fn drain_messages() → Option<(u32, u32)>

Non-blocking try_recv() loop. Stale Loaded messages (path doesn't match loading_path) are discarded. Returns resize dimensions as a return value.

5.15 Rendering functions

- **fit_scale()** — min(win_h/img_h, win_w/img_w). Always used since zoom is always 1.0.
- **nearest()** — point sample: one source pixel at the integer-truncated coordinate. Fast and correct at fit-to-window scale.
- **composite()** — alpha-blends a straight-RGBA pixel over BG. Inner blend() shares the formula across R, G, B.
- **render()** — takes Option<View>. Fills with BG if None. Always paints unconditionally — no dirty flag — so that off-screen regions paint correctly when the window is partially off-screen.

5.16 Native dialogs

Both dialogs call MessageBoxW — blocking, correct for modal confirmation. UTF-16 via encode_wide().chain(Some(0)). #[cfg(not(windows))] fallbacks for cross-platform compilation.

5.17 Window helpers

work_area() returns usable desktop dimensions. On Windows calls SPI_GETWORKAREA for the exact taskbar-excluded rect; elsewhere uses 90%×85% heuristic. Separated into a #[cfg(windows)] helper returning Option so the fallback path is clean.

clamp_to_work_area() scales image dimensions to fit the usable area. **center_window()** positions the window at the monitor centre with a minimum Y offset of 30×scale_factor pixels so the title bar is always reachable.

5.18 fn main() — startup sequence

- **image_size_hint()** reads only the file header for dimensions (no full decode) — fast even on a cold start. Used to size the window before decoding begins.
- **with_visible(!start_hidden)** — window is hidden when an image argument is provided. This prevents any intermediate state being visible. When started without an argument, the window is visible immediately as a drag-and-drop target.
- **Synchronous gray paint** — after creating the softbuffer surface, fill it with BG gray and call present() before the event loop starts. This overwrites the DWM white background immediately.
- **Background thread decode** — open_image() spawns a thread for the full decode. The gray window is visible while it loads.
- **Reveal on pending_center** — the window is made visible inside the UserEvent(WakeUp) handler, specifically in the pending_center branch. By that point the resize has been applied and center_window() has run, so the window appears already correctly sized and positioned with the image painted.
- **RedrawRequested always paints** — no dirty flag. This ensures that when the OS exposes a region that was off-screen (e.g. after dragging the window back fully on-screen), the pixels are always correct rather than showing stale content from a previous image.
- **GIF control flow** — ControlFlow::WaitUntil(next_frame_at) when animating; ControlFlow::Wait otherwise. Zero CPU when idle.

6. Threading & Asynchronous Design

6.1 Why threads?

Image decoding is I/O-bound and CPU-bound. A large image can take hundreds of milliseconds. Doing this on the event loop thread freezes the window.

6.2 The channel pipeline

- One bounded crossbeam channel, capacity 32.
- msg_tx cloned for each background thread — cheap.
- msg_rx drained in drain_messages() on each WakeUp.
- Background threads post a Msg then call proxy.send_event(WakeUp).

■ *proxy.send_event() is thread-safe — EventLoopProxy is Arc-based.*

6.3 ControlFlow::Wait — zero idle CPU

The event loop sleeps until an event arrives. ControlFlow::WaitUntil wakes precisely at the next GIF frame deadline. When idle, the process uses effectively zero CPU.

6.4 Header probe + async decode

image_size_hint() reads only the file header (a few bytes) to get dimensions. The window is created and shown at the correct size with a gray background. The full decode runs on a background thread. This keeps startup fast even on cold Windows process starts (binary evicted from page cache) because the header read is nearly instant.

6.5 Window reveal sequence

When an image argument is provided: window is hidden → gray paint → decode thread runs → WakeUp arrives → resize issued → pending_center=true → next WakeUp: center_window() → set_visible(true) → RedrawRequested paints the image. The user sees the window appear already showing the image at the correct size and position.

6.6 Stale result discarding

Loaded messages are checked against loading_path. Mismatches are discarded, preventing the wrong image appearing when the user navigates faster than images decode.

6.7 Async deletion

remove_file() runs on a background thread. The deleting flag prevents double-deletion. The result arrives as Msg::Deleted.

6.8 Off-screen pixel correctness

RedrawRequested always paints — no dirty flag. When the OS exposes a region that was clipped because the window was partially off-screen, the next RedrawRequested writes fresh pixels for the entire window, preventing stale content from a previous image appearing in that region.

6.9 The two-event resize/centre sequence

window.request_inner_size() is asynchronous. Calling center_window() in the same handler reads the pre-resize size and computes the wrong position. Fix: set pending_center=true on the WakeUp that issues resize. On the next WakeUp the resize has landed and center_window() is accurate.

7. Key Design Decisions & Trade-offs

winit + softbuffer over egui/eframe

egui/eframe is a full GUI toolkit with font atlas, widget layout, and OpenGL backend — ~200+ crates and ~9 MB for a viewer displaying one image. winit + softbuffer: ~50-70 crates and ~2-3 MB. Trade-off: no widgets, native OS dialogs required.

Frame and DecodedImage in loader.rs

Eliminates the RawImage→DecodedImage conversion that existed when types were split across files. load_image() returns DecodedImage directly.

No zoom, nearest-neighbour only

Scroll-wheel zoom was implemented and then removed at the user's request. Removing it eliminated ZOOM_MIN/MAX, zoom_at(), bilinear(), lerp_u8(), View.zoom, and the use_bilinear branch. Nearest-neighbour is correct and fast at fit-to-window scale.

Single-image navigation is a no-op

nav.rs next() and prev() return None when files.len() < 2. Previously they wrapped to the same path and triggered a redundant reload.

No dirty flag — always paint on RedrawRequested

A dirty flag prevented unnecessary repaints but also prevented the OS from refreshing stale off-screen regions when the window moved back fully on-screen. Always painting is correct and cheap since ControlFlow::Wait means RedrawRequested only fires when explicitly requested or when the OS needs a repaint.

Header probe for fast startup

Reading only the image header (a few bytes) for dimensions avoids a full decode before the window appears. On a cold Windows start where the binary and libraries are being loaded from disk, the full synchronous decode could take a minute. The gray window appears instantly; the image arrives asynchronously.

Hidden window until image is ready

with_visible(false) when an image argument is provided. The window is revealed only after resize and center are applied. The user sees the window appear already showing the correct image with no intermediate states. When started without an argument, the window is visible immediately as a drag-and-drop target.

Return values over side effects

apply_image() returns Option<(u32,u32)> and drain_messages() propagates it. The App struct has no window-management fields except pending_center.

Native OS dialogs

MessageBoxW is blocking — correct for modal confirmation. No rendering code, inherits system theme, eliminates the need to suppress navigation input while the dialog is open.

8. Caveats & Known Limitations

Startup white flash (minimised, not eliminated)

DWM fills every new window with white before the application paints. Mitigated by: (1) hiding the window with with_visible(false) when an image is provided, and (2) painting gray synchronously before the event loop starts. The window the user sees is always already gray or showing the image. The DWM white is never visible in normal operation.

Cold-start latency

On Windows, if the executable hasn't been run for a long time, the OS may have evicted the binary and all DLLs from the page cache. The loader must re-read them from disk, verify Authenticode signatures, and initialise the CRT — which can take up to a minute on slow storage. This is a Windows OS behaviour, not a code issue. Subsequent runs are fast because everything is in the page cache.

No status bar

Filename, index, and dimensions are not displayed. Adding them requires a font rasteriser crate.

Animated GIF only

APNG and animated WebP/AVIF would need separate decoder paths.

Render loop is O(win_w × win_h)

The renderer iterates every window pixel on every redraw. Fast enough but could be parallelised with rayon for very large windows.

No pan persistence across images

Pan resets to centre on every image navigation.

JXL HDR clamped, not tone-mapped

f32_to_u8 clamps values above 1.0. A tone mapping operator would produce better results for HDR JXL content.

SVG external resources not loaded

usvg with default options does not resolve external fonts or linked images.

monitor_usable heuristic on non-Windows

The SPI_GETWORKAREA path is Windows-only. Other platforms use 90%×85%.

9. Installer — main.wxs

The installer is built with WiX 3 via the cargo-wix subcommand. Run cargo wix from the project root to produce an MSI in target\wix\. WiX 4/5 would be cleaner but cargo-wix only supports WiX 3.

9.1 Product identity

The **UpgradeCode** GUID is fixed and must never change — it is what Windows uses to detect that a new version is an upgrade of the same product rather than a different application. The **Product Id='*'** generates a new GUID on every build, which is correct for major upgrades. **MajorUpgrade** ensures that installing a new version automatically removes the old one.

9.2 Codepage constraints

WiX 3's linker encodes strings into the MSI database using the codepage declared in Product/@Codepage and Package/@SummaryCodepage. The MSI summary information stream only accepts ANSI code pages — UTF-8 (65001) is not permitted there. Both are set to 1252, and the manufacturer name is ASCII-transliterated ('Hizlikan' rather than 'H■zl■kan') to avoid LGHT0311 errors. This is a WiX 3 limitation; WiX 4 handles UTF-8 correctly.

9.3 File associations — why explicit RegistryValue elements

WiX 3's built-in ProgId/Extension elements generate registry entries internally and can produce duplicate primary keys (LGHT0130) when registering multiple extensions under one ProgId. The collision is a known WiX 3 bug. The workaround is to write all registry entries as explicit RegistryValue elements, which gives full control over the Registry table and no internal collisions.

9.4 Three installer features

The feature tree presents three optional features, all checked by default:

Feature	Default	What it does
Add to Open With	✓ On	Adds Devoid Viewer to the right-click Open With menu for all supported formats witho
Set as Default Image Viewer	✓ On	Makes Devoid Viewer the default opener for all supported formats, replacing the curre
Desktop Shortcut	✓ On	Places a shortcut on the desktop.

The ProgId registration (DevoidViewer.ImageFile key, launch command, and icon) is always installed as part of the core application since both association features depend on it.

9.5 Registry structure

Three components write to the registry:

- **ProgIdRegistration** — writes HKCR\DevoidViewer.ImageFile (display name), HKCR\DevoidViewer.ImageFile\shell\open\command (launch command with "%1"), and HKCR\DevoidViewer.ImageFile\DefaultIcon (icon path). Always installed.

- **OpenWithAssociations** — writes HKCR\.ext\OpenWithProgIds\DevoidViewer.ImageFile for each of the 11 supported extensions. This registers Devoid Viewer in the Open With list without changing the default.
- **DefaultAssociations** — writes HKCR\.ext = DevoidViewer.ImageFile for each extension, making Devoid Viewer the default opener.

WiX automatically removes all registry entries on uninstall since they are owned by components that are removed when the product is uninstalled.

9.6 Shell icon cache refresh

A CustomAction runs ie4uinit.exe -ClearIconCache after install and uninstall so that Explorer immediately shows the correct file type icons without requiring a reboot. It is Execute='immediate' (not deferred) because it has no rollback requirement and does not need to run inside the install transaction.

9.7 Building the installer

Prerequisites: WiX Toolset 3.x installed with the WIX environment variable set, and cargo-wix installed (cargo install cargo-wix). Then:

- cargo wix — builds the release binary and produces the MSI in target\wix\.
- cargo wix --nocapture — shows full candle/light output for debugging.

10. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

Borrowing
Lend a reference (&T;) without transferring ownership. References cannot outlive the value they point to.

&mut; self vs &self;
&mut; self = exclusive mutable access. &self; = shared read-only access. The compiler enforces that both cannot coexist.

Option
Either Some(value) or None. Safe replacement for null pointers.

Result
Either Ok(value) or Err(error). The ? operator propagates errors automatically.

Vec
A growable heap-allocated array.

HashMap
A hash table. O(1) average lookup, insert, and delete.

Arc
Atomically Reference Counted smart pointer. Shared ownership across threads.

Enum with data
Rust enums carry data per variant. Msg::Loaded carries path, image, bool. Unlike C enums which carry no data.

From / Into
Standard conversion traits. impl From for Y lets you write x.into().

then_some
(condition).then_some(value) returns Some(value) if true, None otherwise.

Lifetime
A compile-time label on a reference. View<'a> borrows from App — the compiler ensures View cannot outlive App.

Closure
An anonymous function. move |event, elwt| {...} moves ownership of captured variables — required for event_loop.run().

match
Exhaustive pattern matching. Every variant must be handled.

#[derive(...)]
Auto-generates trait implementations. #[derive(Clone, Debug)] writes clone() and the debug formatter.

Send
Marker trait: safe to transfer to another thread. Derived automatically for types containing only Send fields.

ControlFlow
Winit enum. Wait sleeps until an event. WaitUntil sleeps until a deadline. Poll spins continuously.

EventLoopProxy
Thread-safe handle to the event loop. proxy.send_event() injects a synthetic event from any thread.

NonZeroU32
A u32 guaranteed non-zero at compile time. softbuffer requires this for resize.

cfg(windows)
Conditional compilation. Code in #[cfg(windows)] blocks compiles on Windows only.

unsafe
Allows calling C API functions. Used for MessageBoxW and SystemParametersInfoW.

type alias
type Proxy = ...; creates a shorter name with no runtime cost.

Zero-size struct
struct WakeUp; has no fields and zero runtime size. Used as a type-level marker.